

1. 01_weather_fetch.py

What it does:

Fetches hourly weather history for each station from Open-Meteo, starting from that station's opening date and ending at the configured latest available date. It enriches each hourly row with station metadata and the weather-grid metadata, including how far the station is from the grid point used for the weather values. This script is the raw weather collection step; later scripts aggregate these hourly observations into monthly station-level weather features

Inputs:

data/sites.csv

Outputs:

transformed_data/weather_outputs/station_weather_hourly.csv

checkpoint and failure logs in transformed_data/weather_outputs/

2. 02_crash_preprocess_match_bike_radII.py

What it does:

Loads the raw crash data , filters it down to crashes involving a bicycle road user, converts the crash coordinates into latitude/longitude, and spatially matches each crash to nearby stations. It builds the core spatial link between crashes and stations by checking whether each crash falls within chosen distance thresholds around each station. For the current pipeline, the important output is the 1 km bike-only matching, which becomes the crash target source for later monthly modeling.

Inputs:

data/OPENDATA_MAP_2017-2024.xlsx

data/sites.csv

Outputs used by the current pipeline:

transformed_data/crash_outputs/crashes_bike_only.csv

transformed_data/crash_outputs/crashes_bike_only_matched_to_sites_1km.csv

transformed_data/crash_outputs/site_bike_crash_summary_1km.csv

3. 03_build_monthly_bike_crash_datasets.py

What it does:

Reads the raw bike-count files and aggregates the interval-level traffic data into one row per station per month, with summary statistics such as totals, means, maxima, minima, and zero-count intervals. It also aggregates the hourly weather into monthly weather summaries and merges those with the monthly traffic data. Finally, it joins in the matched crash counts so the result is a clean station-month dataset that contains traffic, weather, seasonality, and observed crash outcomes together.

Inputs:

raw monthly bike count files in data/

transformed_data/weather_outputs/station_weather_hourly.csv

transformed_data/crash_outputs/crashes_bike_only_matched_to_sites_1km.csv

Main output:

transformed_data/model_outputs/station_month_bike_crash_dataset_1km.csv

4. 04_build_next_month_bike_crash_dataset_1km.py

What it does:

Takes the station-month dataset and converts it into a forecasting dataset by building lagged and rolling historical features for traffic, weather, and crash history. It creates the next-month prediction target so that each row at month t is used to predict whether a crash happens in month $t+1$. This is the step that turns the problem from descriptive monthly monitoring into a proper forward-looking prediction task.

Input:

transformed_data/model_outputs/station_month_bike_crash_dataset_1km.csv

Output:

transformed_data/model_outputs/station_month_bike_crash_dataset_1km_next_month.csv

5. 05_train_optuna_catboost_1km.py

What it does:

Runs an Optuna hyperparameter search for CatBoost using the 1 km next-month forecasting dataset and a time-based validation split. It evaluates many CatBoost parameter combinations and keeps track of which settings produce the strongest predictive performance on later unseen months. After the tuning run, it writes out the best parameter set and validation summary so the grouped model export step can reuse that tuned configuration instead of relying on default settings.

Input:

transformed_data/model_outputs/station_month_bike_crash_dataset_1km_next_month.csv

Outputs:

transformed_data/model_outputs/optuna_catboost_1km_study.csv

transformed_data/model_outputs/optuna_catboost_1km_best_params.json

transformed_data/model_outputs/optuna_catboost_1km_best_metrics.json

transformed_data/model_outputs/optuna_catboost_1km_summary.txt

6. 06_build_grouped_station_datasets.py

What it does:

Creates grouped counter entities by merging nearby related stations that likely represent the same local place rather than truly separate safety locations. It uses station naming patterns, municipality, and geographic proximity to decide which stations belong together, then rebuilds the monthly and next-month datasets at the grouped level. This changes the unit of analysis

from individual counters to grouped counters, which reduces duplication and can improve interpretability and model stability.

Inputs:

data/sites.csv

transformed_data/model_outputs/station_month_bike_crash_dataset_1km.csv

transformed_data/crash_outputs/crashes_bike_only_matched_to_sites_1km.csv

Outputs:

transformed_data/combined_station_groups_1km/outputs/group_mapping.csv

transformed_data/combined_station_groups_1km/outputs/station_group_month_bike_crash_dataset_1km.csv

transformed_data/combined_station_groups_1km/outputs/station_group_month_bike_crash_dataset_1km_next_month.csv

transformed_data/combined_station_groups_1km/outputs/group_dataset_summary.txt

7. 07_export_grouped_app_model.py

What it does:

Trains the final grouped CatBoost classifier that will actually be used by the app, using the grouped next-month dataset produced in the prior step. It evaluates performance on a held-out later-period split, chooses a probability threshold, and then retrains the final deployable model on the full grouped dataset. It also saves metadata such as feature order, fill values, metrics, and feature importance so the app can reconstruct prediction inputs consistently.

Inputs:

transformed_data/combined_station_groups_1km/outputs/station_group_month_bike_crash_dataset_1km_next_month.csv

transformed_data/model_outputs/optuna_catboost_1km_best_params.json

Outputs:

transformed_data/combined_station_groups_1km/models/next_month_catboost_grouped_bike_1km.cbm

transformed_data/combined_station_groups_1km/models/next_month_catboost_grouped_bike_1km_metadata.json

transformed_data/combined_station_groups_1km/models/next_month_catboost_grouped_bike_1km_feature_importance.csv

transformed_data/combined_station_groups_1km/models/deployed_grouped_model_summary.json

8. 08_build_grouped_app_assets.py

What it does:

Transforms the grouped modeling outputs into app-ready files for visualization and interaction. It builds the historical ranking, the forecast ranking, the per-group baseline feature rows used for live prediction, the editable control ranges, and the bootstrap summary used to initialize the UI. This is the bridge between modeling and presentation, turning model artifacts into files the frontend can load directly.

Inputs:

transformed_data/combined_station_groups_1km/outputs/station_group_month_bike_crash_dataset_1km.csv

transformed_data/combined_station_groups_1km/outputs/station_group_month_bike_crash_dataset_1km_next_month.csv

transformed_data/combined_station_groups_1km/outputs/group_mapping.csv

grouped model files from transformed_data/combined_station_groups_1km/models/

Outputs:

transformed_data/combined_station_groups_1km/app_assets/bootstrap.json

transformed_data/combined_station_groups_1km/app_assets/historical_ranking.csv

transformed_data/combined_station_groups_1km/app_assets/forecast_ranking.csv

transformed_data/combined_station_groups_1km/app_assets/group_baselines.csv

transformed_data/combined_station_groups_1km/app_assets/group_metadata.csv

transformed_data/combined_station_groups_1km/app_assets/control_ranges.json

9. 09_app_grouped.py

What it does:

Runs the grouped Flask web app and exposes the grouped rankings and prediction engine through a map-based interface. At startup, it loads the prebuilt grouped app assets and the saved grouped CatBoost model into memory so the app can respond quickly without retraining anything. It serves the main page, provides station/group detail data to the frontend, and returns live prediction results when a user adjusts inputs and asks for a forecast.

Inputs:

grouped app assets from transformed_data/combined_station_groups_1km/app_assets/

grouped model files from transformed_data/combined_station_groups_1km/models/

templates/index_grouped.html

static/grouped_app.js

static/grouped_styles.css

Output:

local web app at http://127.0.0.1:5000